



TP Final: Análisis y Pronóstico de BTC con Enfoque Funcional

Análisis y pronóstico del precio de Bitcoin

PROGRAMACIÓN FUNCIONAL · LICENCIATURA EN CIENCIA DE DATOS

Alumnos	Abraham Milena, Arguinzoniz Julieta, Juárez María Ailén, Muñoz Tadeo, Rossello Nicolás
Comisión	70AT
Trabajo práctico	Análisis y pronóstico de BTC
Fecha de entrega	18 de septiembre de 2026
Docente	Javier Epeloa

1 Introducción

La pregunta que nos guió fue simple: ¿se puede pronosticar el precio de Bitcoin a una semana? Bitcoin se mueve muchísimo, y eso lo hace un buen caso para poner a prueba un modelo. Cualquiera puede dibujar una curva que pase cerca de los datos que ya tiene; lo difícil es acertar los días que todavía no vio.

Trabajamos con el archivo `btc-timeseries.json`: 365 precios diarios seguidos, donde cada precio es el promedio entre el máximo y el mínimo del día. Con esos datos armamos dos modelos y los comparamos contra el pronóstico más básico posible, que es suponer que mañana el precio va a ser igual que hoy:

1. **Ventana móvil ($W = 7$):** pronostica el promedio de los últimos siete días.
2. **Función no lineal (Tanh + Fourier):** una curva que mezcla una tangente hiperbólica con ondas seno y coseno, ajustada a los datos.

2 Objetivos

2.1 Objetivo general

Armar con programación funcional un pipeline que pronostique el precio de Bitcoin y comparar un modelo de ventana móvil y uno no lineal contra el pronóstico ingenuo de repetir el último precio.

2.2 Objetivos específicos

1. Medir si el precio y sus retornos son estacionarios (test ADF), cuánto se parecen los retornos de un día al siguiente y cuál es la volatilidad anual.
 2. Implementar los tres modelos usando configuración inmutable, funciones de orden superior, un tipo de resultado con `map` para los ajustes que pueden fallar y `reduce` para juntar resultados, y comprobar con `assert` las leyes del Functor.
 3. Comparar el error de los tres modelos en 30 ventanas de validación de 7 días cada una y ver con la prueba de Wilcoxon si las diferencias son reales.
 4. Medir cuánto empeora el modelo no lineal cuando tiene que pronosticar días que no vio, comparado con su error sobre los datos que usó para ajustarse.
-

3 Metodología

3.1 Datos y herramientas

Datos. El archivo tiene 365 días seguidos, del 29 de agosto de 2025 al 28 de agosto de 2026, sin huecos ni repetidos. Cada registro trae un índice `t`, la fecha y el precio, que es el promedio entre el máximo y el mínimo de ese día. Hay algo a tener en cuenta: no es el precio de cierre sino un promedio del día, y promediar puede hacer que un día se parezca al siguiente más de lo que se parecería con el cierre.

Las herramientas. Python 3.12 con NumPy 2.4, pandas 3.0, SciPy 1.17 (`curve_fit` y `wilcoxon`), statsmodels 0.15 (`adfuller` y `acf`) y Matplotlib 3.10, más `functools` , `dataclasses` y `typing` de la librería estándar. Son las versiones con las que corrimos el notebook completo. Por la consigna, dejamos afuera scikit-learn y cualquier modelo de Machine Learning. Las métricas de error (MAE, MAPE y RMSE) las escribimos nosotros como funciones puras.

3.2 Paso a paso

1. **Cargar los datos.** Leemos el JSON, lo pasamos a una tabla y calculamos los retornos diarios, es decir, cuánto subió o bajó el precio de un día al siguiente en términos relativos. Usamos la versión logarítmica, que es la habitual en finanzas porque permite sumar los retornos de varios días.
2. **Diagnosticar la serie.** Corremos el test ADF sobre el precio y sobre los retornos, calculamos la autocorrelación de los retornos hasta 10 días atrás y la volatilidad. Para pasarla a anual consideramos 365 días y no los 252 días hábiles que se usan con las acciones, porque Bitcoin cotiza todos los días, fines de semana incluidos.
3. **Armar las ventanas de prueba.** Definimos 30 ventanas: arrancamos con 150 días para ajustar, pronosticamos los 7 siguientes, avanzamos 7 días y repetimos. Los puntos de corte quedan en una tupla, así nadie los puede modificar por accidente. Las definimos antes de tocar cualquier modelo para que los tres se evalúen exactamente sobre los mismos días; si no, no se podrían comparar de a pares.
4. **Modelo base ($W = 1$).** Repite el último precio durante los 7 días.
5. **Ventana móvil ($W = 7$).** Una función recibe el tamaño de ventana y devuelve otra función, que promedia los últimos 7 precios con `reduce` y repite ese promedio durante los 7 días. El

promedio queda fijo: no se recalcula usando sus propios pronósticos.

6. **Función no lineal.** Ajustamos con `curve_fit` una curva que suma dos partes. La primera es una tangente hiperbólica, una curva con forma de S que representa la tendencia general: el precio sube o baja y después se estabiliza. La segunda son cuatro pares de ondas seno y coseno, que representan las subidas y bajadas que se repiten. En total la curva tiene 12 parámetros, y `curve_fit` los ajusta para que pase lo más cerca posible de los datos. Como no sabíamos cada cuánto se repiten las ondas, probamos cinco opciones (el doble del período de ajuste, el período completo, la mitad, un tercio y un cuarto) y nos quedamos con la que mejor se ajustaba. Si ninguna converge, la función devuelve un resultado de error en vez de frenar todo.
7. **Probar la extrapolación.** Ajustamos la función no lineal con los primeros 335 días y vemos qué tan bien pronostica los últimos 30, que nunca vio.
8. **Comparar los modelos.** Evaluamos las 30 ventanas con `map` y juntamos los resultados con `reduce`. En cada ventana medimos el error en dólares (MAE) y en porcentaje (MAPE), y comparamos cada modelo contra el base con la prueba de Wilcoxon. Como hacemos dos comparaciones, bajamos el umbral de significancia de 0,05 a 0,025 (corrección de Bonferroni), para no encontrar diferencias por pura casualidad.
9. **Proyectar a futuro.** Ajustamos todo con los 365 días, proyectamos 30 días hacia adelante y dibujamos una banda alrededor del último precio que muestra hasta dónde es razonable que se mueva, según la volatilidad histórica. La banda se va abriendo con los días, porque cuanto más lejos miramos, más incertidumbre hay.

¿Por qué este orden? Porque el diagnóstico es lo que justifica elegir el modelo base: si el precio no es estacionario, repetir el último valor es el punto de partida natural, y cualquier modelo más complejo tiene que ganarle para valer la pena.

3.3 Decisiones que tomamos

Decisión	Qué elegimos	Por qué
Contra qué comparar	Repetir el último precio ($W = 1$)	Si el precio no es estacionario, es lo mínimo que un modelo tiene que superar. Si no le gana, no se justifica que sea más complicado.
Tamaño de la ventana	7 días	Es una semana entera de cotización, fines de semana incluidos. Solo probamos este tamaño; lo retomamos en las limitaciones.
Horizonte y avance	7 días cada uno	Si avanzamos lo mismo que pronosticamos, las ventanas de prueba no se pisan y ningún día cuenta dos veces.
Días iniciales para ajustar	150	Son suficientes para estimar los 12 parámetros de la función no lineal y todavía entran 30 ventanas de prueba en el año.
Cómo medir el error	MAE y MAPE	El MAE se entiende directo en dólares. El MAPE, en porcentaje, permite comparar ventanas donde el precio estaba en 60 mil con otras donde estaba en 120 mil.
Prueba estadística	Wilcoxon y no t de Student	Los errores están muy desparejos: en el modelo base el promedio es 3,44 % pero la mediana es 2,54 %, porque

Decisión	Qué elegimos	Por qué
		unas pocas semanas malas tiran el promedio para arriba. No podemos suponer normalidad.
Qué hacer si un ajuste falla	Devolver un <code>FitOutcome</code> con <code>map</code>	Que <code>curve_fit</code> no converja es algo que puede pasar, no un error del programa. Lo tratamos como un dato más y el pipeline sigue. Solo atrapamos los errores de la optimización, para que un bug de verdad no quede escondido.
Cómo armar la función no lineal	Una función que fabrica funciones (closure)	Cada período candidato genera su propia función, sin compartir nada con las otras. Eso nos deja probar todos los candidatos con un simple <code>map</code> .
Cómo juntar los resultados	<code>reduce</code> sobre tuplas	Unir tuplas cumple las propiedades de un Monoid (se puede agrupar en cualquier orden y la tupla vacía no cambia nada), así que el <code>reduce</code> es seguro. Nos evita una lista que va creciendo con <code>append</code> .
Carga de datos	pandas	Por comodidad para manejar fechas. Sabemos que no es funcional, pero los modelos reciben tuplas o arrays, no tablas.

Donde no fuimos 100 % funcionales. pandas y NumPy trabajan a su manera, no con funciones puras. La función que evalúa cada ventana lee la serie desde una variable global: no la modifica, pero depende de ella. Y `make_model` usa un `for` que va actualizando una variable local. Ninguna de estas partes cambia nada fuera de sí misma, pero preferimos decirlo.

4 Resultados

4.1 Resumen

Qué medimos	Valor	Comentario
Registros de entrada	365	La serie completa
Registros descartados	0	No había faltantes ni repetidos
Retornos calculados	364	El primer día no tiene día anterior
Ventanas evaluadas	30 de 30	La función no lineal convergió en todas
Leyes del Functor	2 de 2	Comprobadas con <code>assert</code> sobre listas; sobre <code>FitOutcome</code> no las comprobamos

4.2 Cómo es la serie

Indicador	Resultado	Qué dice el test
ADF sobre el precio	estadístico -1,704; p = 0,4292	No se rechaza que tenga raíz unitaria
ADF sobre los retornos	estadístico -6,423; p < 0,0001	Se rechaza que tenga raíz unitaria

Indicador	Resultado	Qué dice el test
Volatilidad diaria	1,84 %	Desvío estándar de los retornos
Volatilidad anual	35,22 %	La diaria llevada a un año de 365 días
Autocorrelación a 1 día	0,35	Entre 2 y 10 días atrás, valores entre $-0,17$ y $0,13$



Figura 1: Precio diario de Bitcoin del 29 de agosto de 2025 al 28 de agosto de 2026. Máximo USD 124.614, mínimo USD 59.205, promedio USD 82.749.

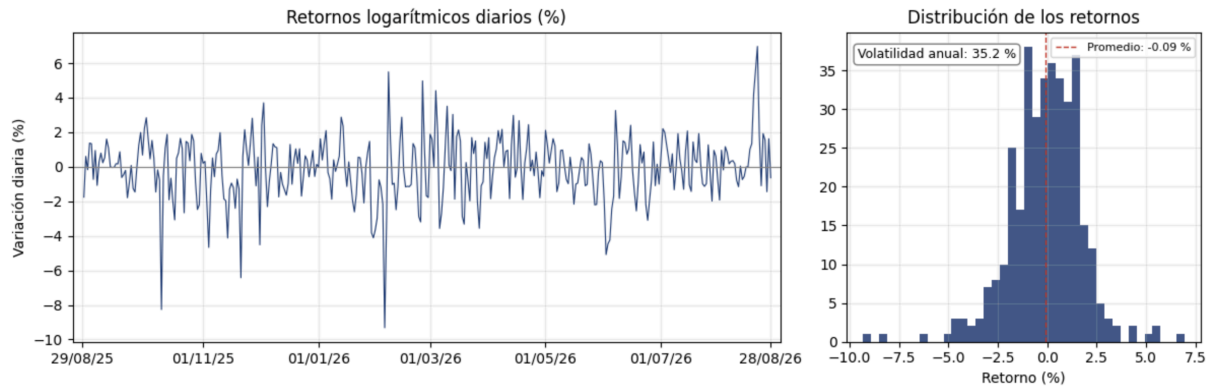


Figura 2: Retornos diarios y su distribución (promedio $-0,09$ %).

4.3 Comparación de los modelos

Modelo	Error promedio (USD)	Error promedio (%)	Error mediano (%)	Ventanas en que ganó el base	Wilcoxon contra el base
Base (W = 1)	2.392,70	3,44 %	2,54 %	—	—
Ventana móvil (W = 7)	2.899,27	4,16 %	2,84 %	18 de 30	p = 0,0473
Función no lineal	7.038,21	10,35 %	8,62 %	25 de 30	p < 0,0001

Con el umbral corregido de 0,025, la diferencia contra la función no lineal queda por debajo y la diferencia contra la ventana móvil queda por encima. La peor semana de cada modelo tuvo un error de 14,09 % (base), 18,60 % (ventana móvil) y 27,33 % (no lineal).

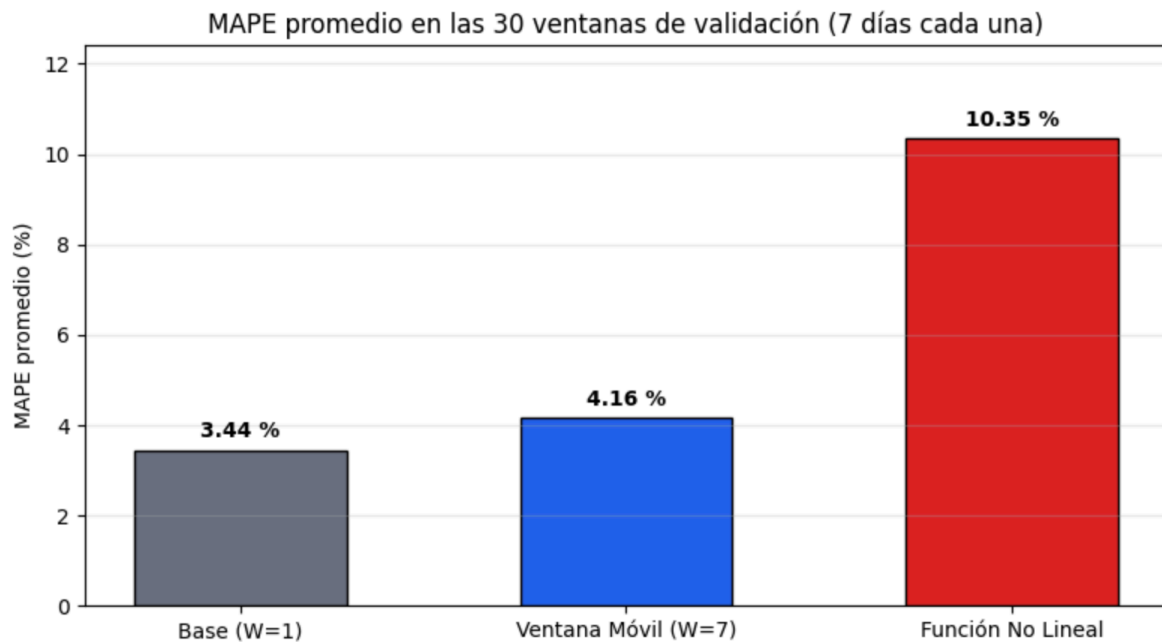


Figura 3: Error promedio de cada modelo en las 30 ventanas.

4.4 La función no lineal fuera de sus datos

Tramo	MAE (USD)	RMSE (USD)	MAPE
Sobre los 335 días que usó para ajustarse	2.501	3.276	3,11 %
Sobre los últimos 30 días, que no vio	44.893	50.898	64,02 %

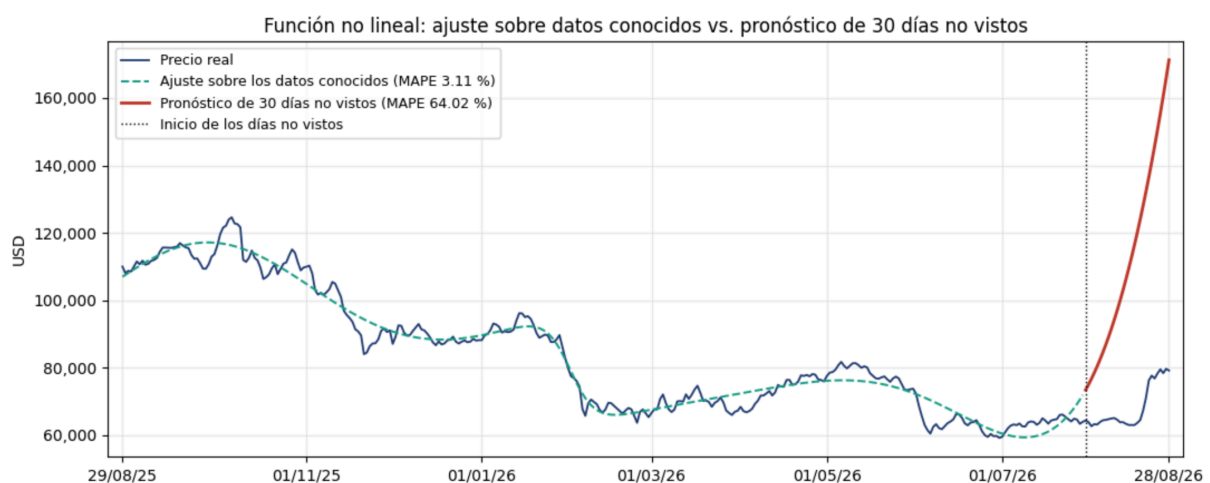


Figura 4: La función no lineal sobre los datos conocidos y pronosticando 30 días que no vio.

4.5 Proyección a 30 días

Modelo	Día 1 (USD)	Día 30 (USD)
Base (W = 1)	79.176,55	79.176,55
Ventana móvil (W = 7)	78.515,01	78.515,01
Función no lineal	86.249,93	352.181,91

La banda de incertidumbre está centrada en el último precio, así que el modelo base siempre queda adentro: no es un mérito, es cómo está construida. La función no lineal se sale de la banda desde el primer día.



Figura 5: Proyección a 30 días de los tres modelos, con la banda de incertidumbre.

4.6 Lo que no salió como esperábamos

- **Tuvimos que descartar el modelo de Machine Learning.** La primera versión usaba Gradient Boosting y quedó afuera por la consigna.
- **El notebook tenía un error escondido.** Cuando sacamos scikit-learn, dos funciones quedaron llamando a código que ya no existía. Como el ajuste no lineal atrapaba cualquier error, el problema aparecía como "curve_fit no convergió", que no era la causa real. El notebook solo parecía andar porque la sesión de Colab tenía guardadas variables de corridas anteriores. Lo arreglamos haciendo que solo se atrapen los errores propios de la optimización, y lo volvimos a correr completo desde cero.
- **Terminamos con dos versiones de la ventana móvil.** En otra rama del repositorio, la ventana usaba sus propios pronósticos para calcular el día siguiente. Esa versión dio un error promedio de 3,79 % y un p-valor de 0,40 contra el base, o sea, ninguna diferencia. Para este informe usamos la versión de promedio fijo.

5 Conclusiones

Objetivo 1 (diagnóstico): cumplido. El test ADF no da evidencia de que el precio sea estacionario ($p = 0,4292$), pero sí la da para los retornos ($p < 0,0001$). Eso respalda usar

"repetir el último precio" como punto de comparación. Lo que nos sorprendió fue la autocorrelación de 0,35 a un día: significa que los retornos no son completamente al azar, así que el precio no es exactamente un paseo aleatorio. Sospechamos que parte de eso viene de cómo se armó el dato, porque promediar el máximo y el mínimo suaviza la serie.

Objetivo 2 (implementación funcional): cumplido a medias. Usamos dataclasses congeladas para la configuración, closures para los modelos, `reduce` sobre tuplas para juntar resultados y un `FitOutcome` cuyo `map` arma la función de pronóstico final. Pero comprobamos las leyes del Functor sobre listas y no sobre `FitOutcome`, que es el Functor que construimos nosotros. Además, `FitOutcome` es un Functor y no una Mónada: tiene `map` pero no `flat_map`, así que no sirve para encadenar varios ajustes que puedan fallar. Y la evaluación de ventanas depende de una variable global, así que solo es pura mientras nadie toque esa serie.

Objetivo 3 (comparación): cumplido. El modelo base fue el que menos se equivocó, tanto en promedio como en mediana. Contra la función no lineal no hay discusión: el base ganó en 25 de las 30 semanas y la prueba es clarísima ($p < 0,0001$). Contra la ventana móvil, en cambio, la diferencia es chica y no muy firme: el p-valor de 0,0473 no llega al umbral corregido de 0,025, el base ganó solo en 18 de 30 semanas y la otra versión de la ventana no mostró ninguna diferencia. Una explicación posible es que promediar la última semana hace que el pronóstico llegue tarde a los cambios, pero con estos datos no podemos afirmarlo.

Objetivo 4 (extrapolación): cumplido. La función no lineal pasó de equivocarse un 3,11 % sobre los datos que usó para ajustarse a equivocarse un 64,02 % sobre los 30 días que no vio: unas 20 veces más. En la proyección llega a USD 352.182 al día 30, cuatro veces el último precio. Fuera de los datos, la curva sigue su propia forma sin nada que la corrija. La lección es que un ajuste que se ve muy bien sobre los datos conocidos no dice nada sobre si sirve para pronosticar.

Qué nos dejó el paradigma funcional. Nos resultó cómodo en el núcleo del trabajo: armar los modelos como funciones configurables, probar variantes con `map`, juntar resultados con `reduce` sin variables intermedias, y comprobar el estimador de ventana con un `assert` de una sola línea. Se volvió incómodo en los bordes. pandas y NumPy tienen su propia lógica, `curve_fit` avisa los fallos con excepciones que tuvimos que convertir en valores, y Python no impide modificar nada, así que la inmutabilidad depende de que uno sea prolijo. El error del notebook lo muestra bien: atrapar todas las excepciones parecía un manejo funcional de errores y terminó escondiendo un bug.

Limitaciones. Usamos un solo activo y un solo año. Treinta ventanas son pocas para detectar diferencias chicas. Probamos un solo tamaño de ventana. El precio es un promedio del día y no el cierre. Y la banda de incertidumbre supone que la volatilidad no cambia.

Qué haríamos después. Probar ventanas de 3, 14 y 30 días; comprobar las leyes del Functor sobre `FitOutcome` y agregarle `flat_map` para que funcione como Maybe/Either; pasarle la serie como parámetro a la evaluación de ventanas; y comparar con un suavizado exponencial.

Anexos y referencias

Referencias

- Apuntes de cátedra de Programación Funcional, Unidades I a IV, sobre todo la Clase 4.